

Computer Security 2026

DOS – Firewalls
Intrusion Detection

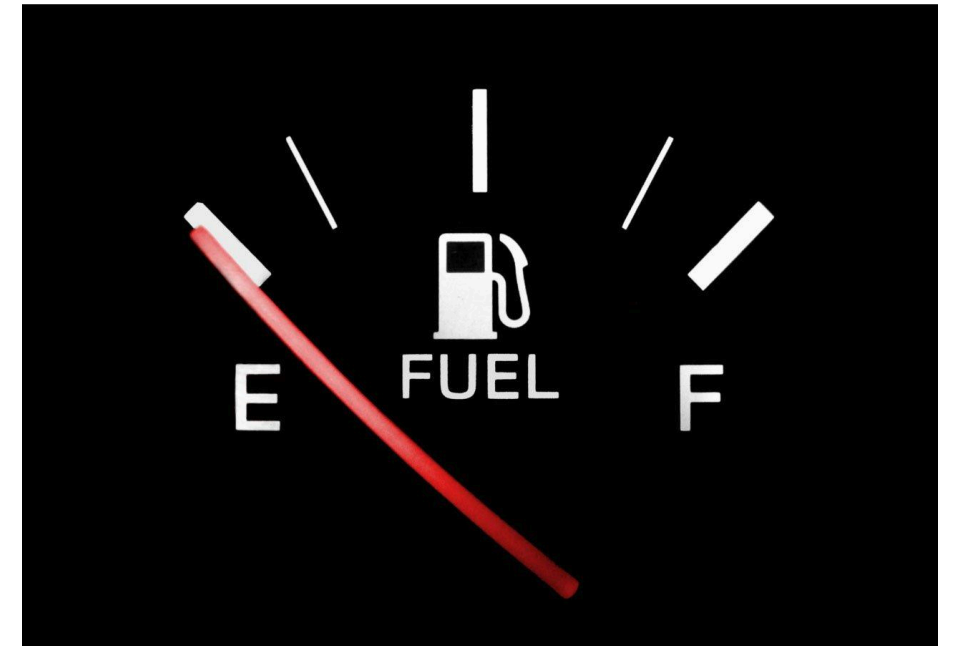
Roberto Guanciale

Denial Of Service

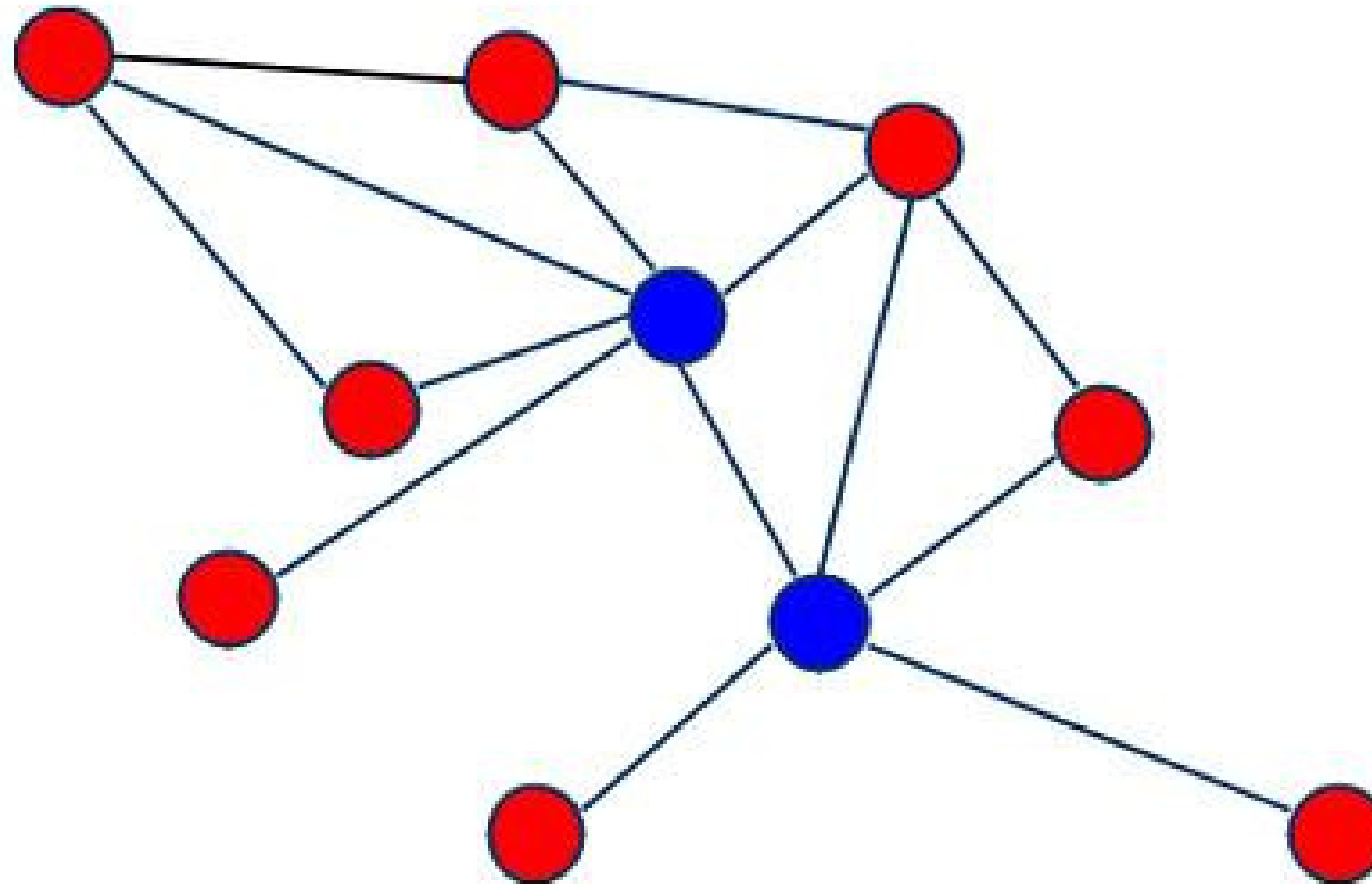
prevents or **impairs** the authorized use of

Networks
Systems
Applications

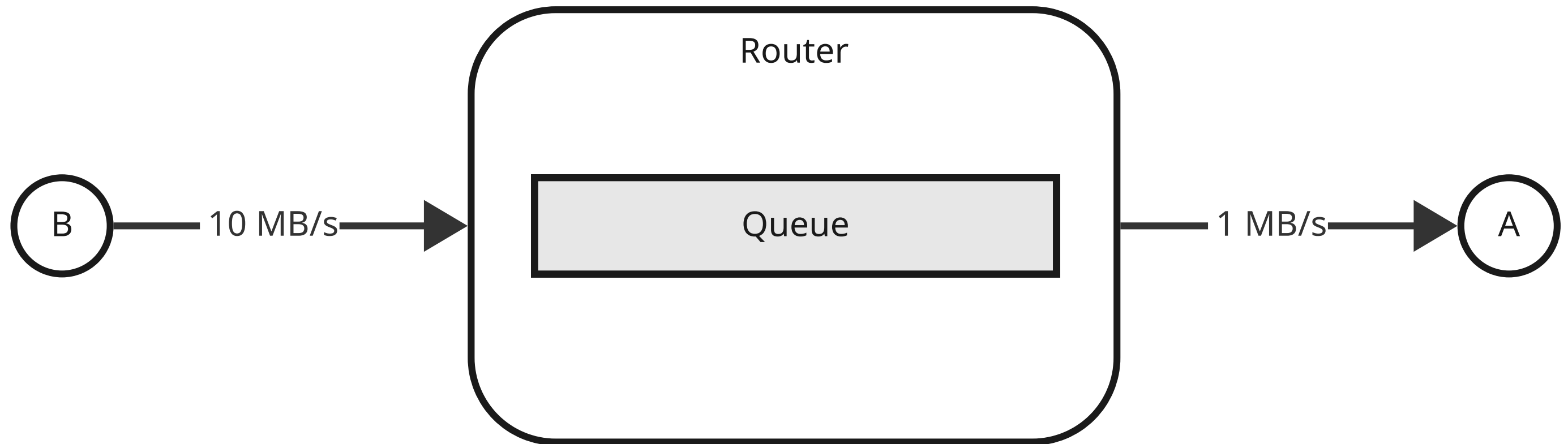
By **exhausting** or **damaging** resource



Networked DOS

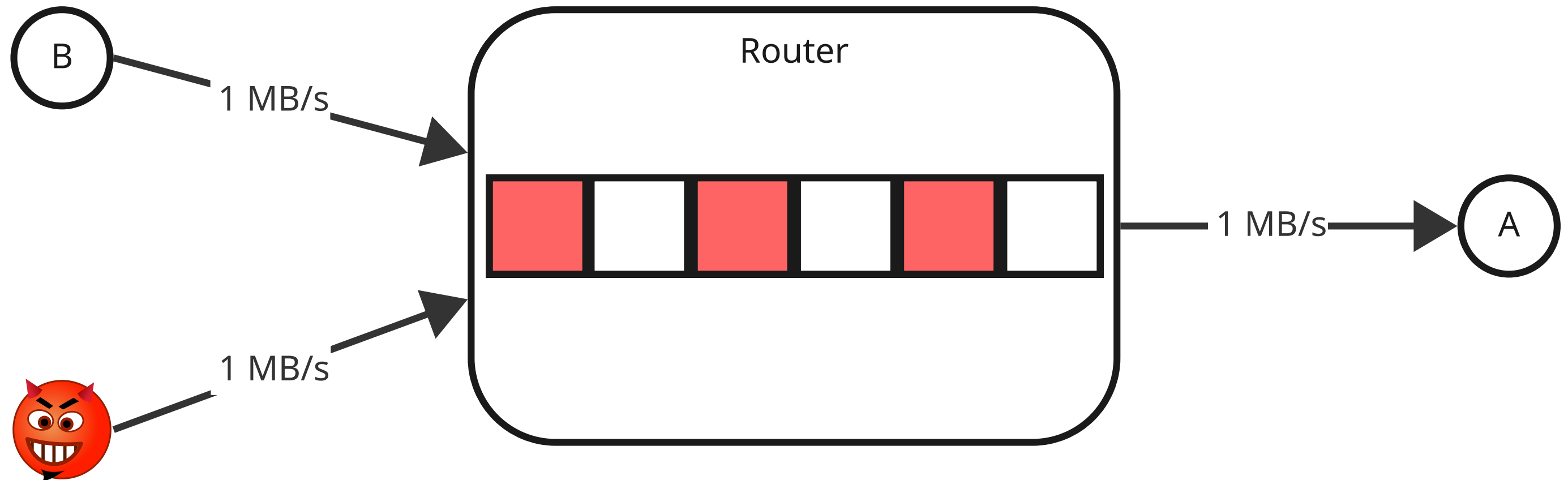


ICMP flood

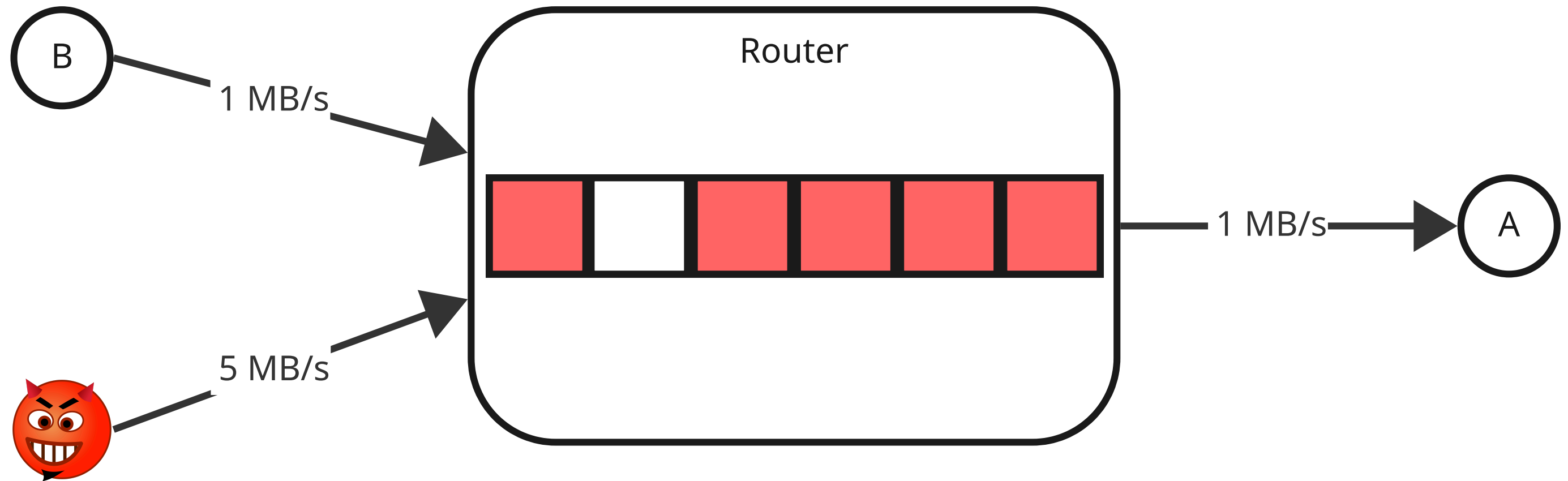


Congestion control via **Congestion Window** of TCP

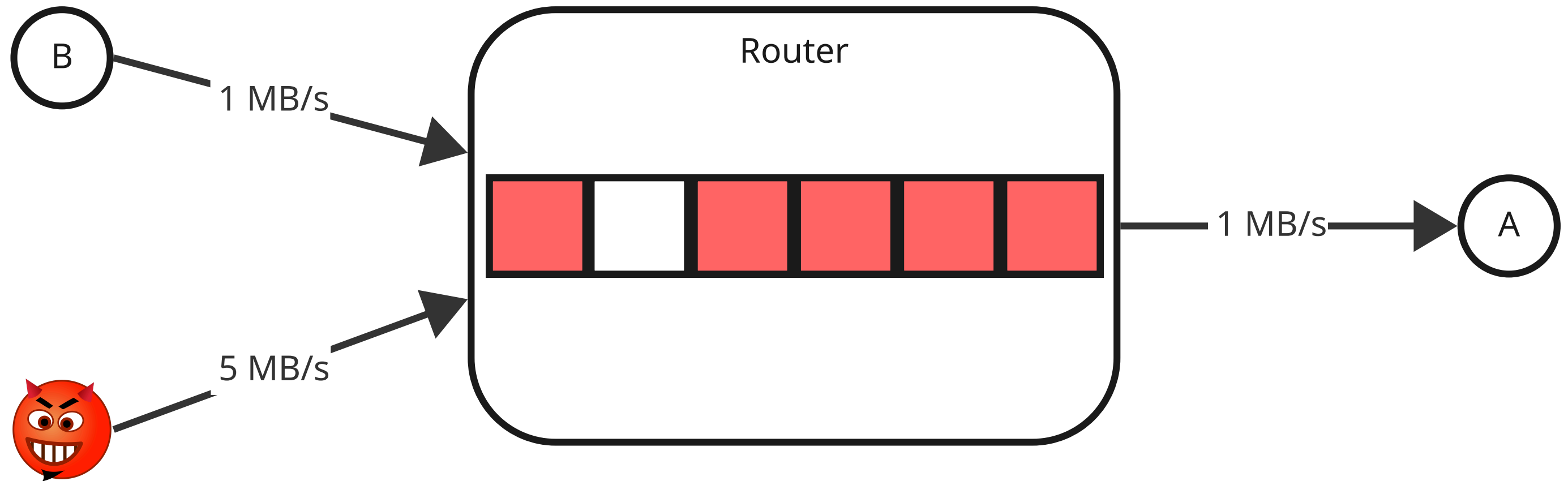
ICMP flood



ICMP flood

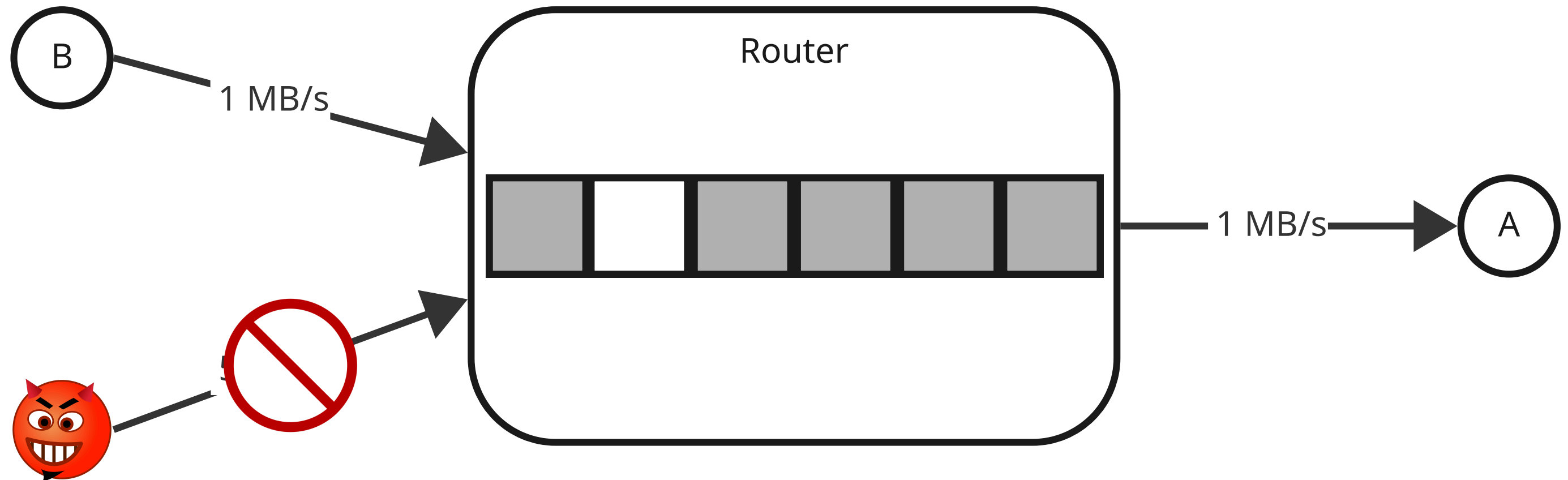


ICMP flood



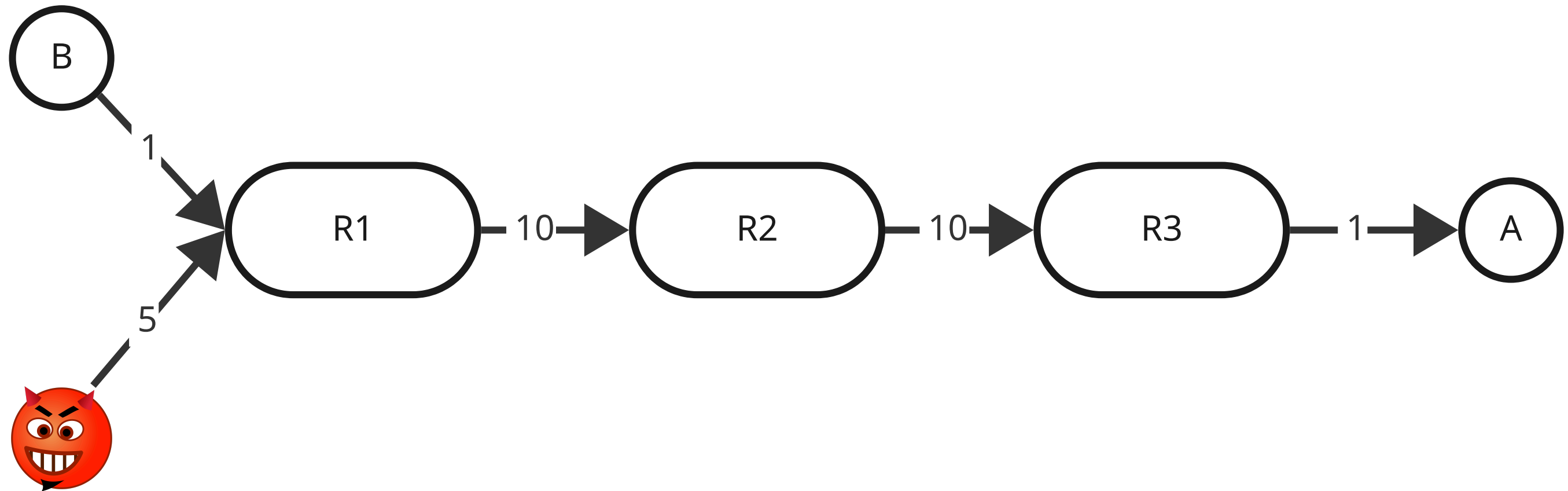
Who can handle this problem?

ICMP flood



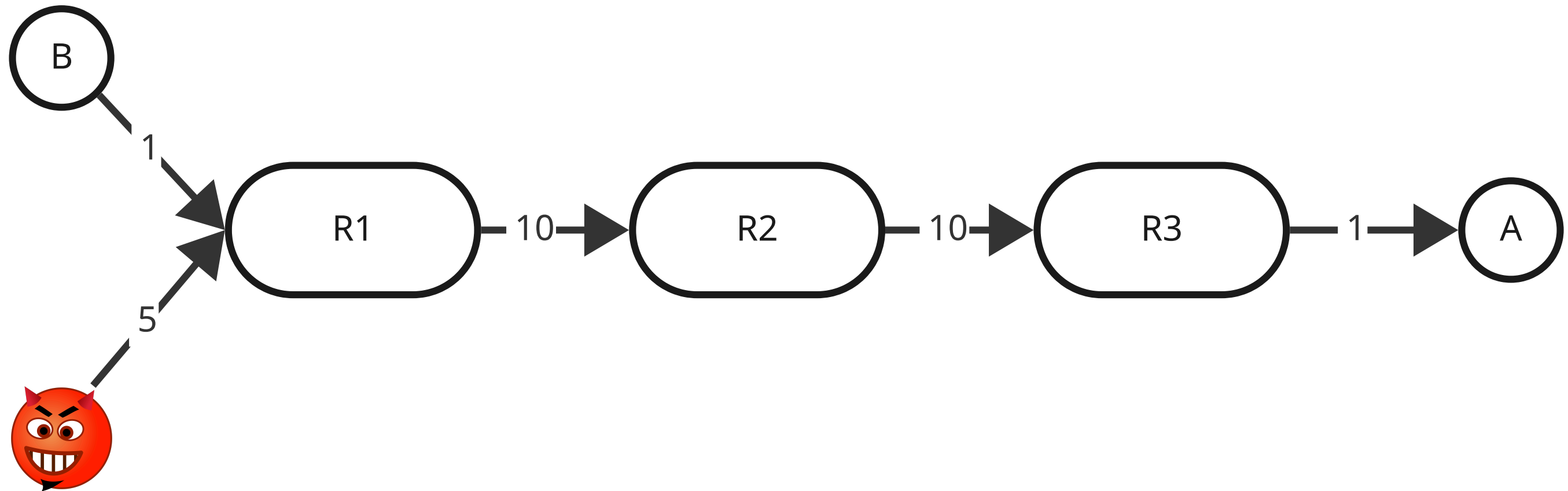
Who can handle this problem?

ICMP flood



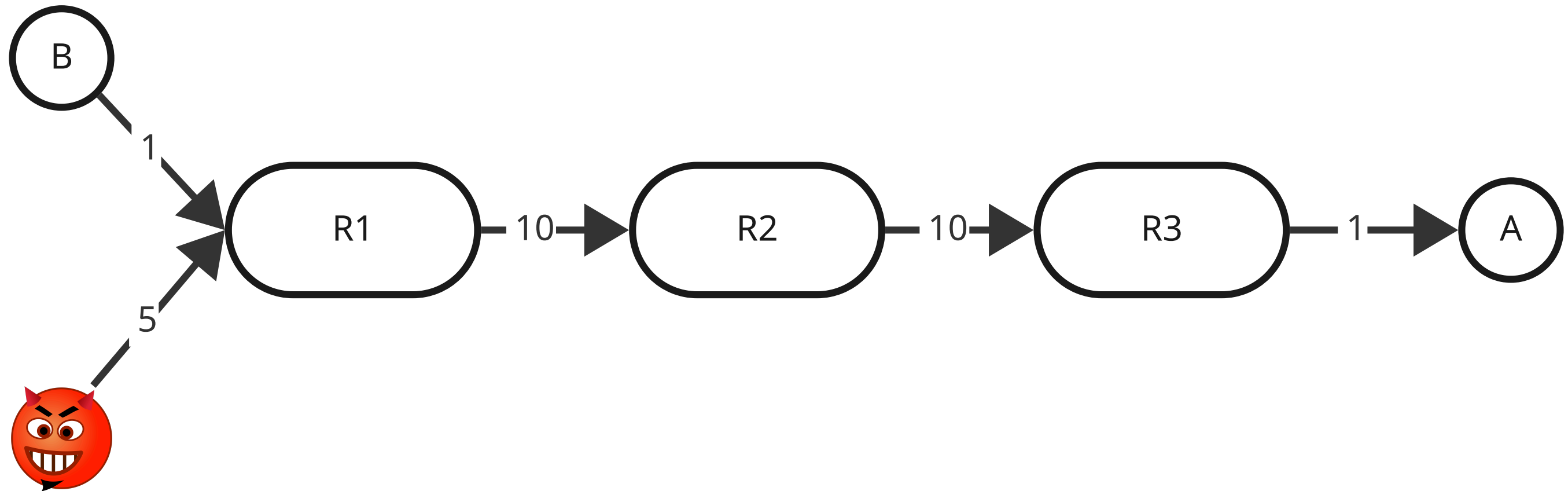
Who can **identify** the problem?

ICMP flood



Who can **stop** the problem?

ICMP flood



Source Address Spoofing: attacker can send packets with srcIP=B

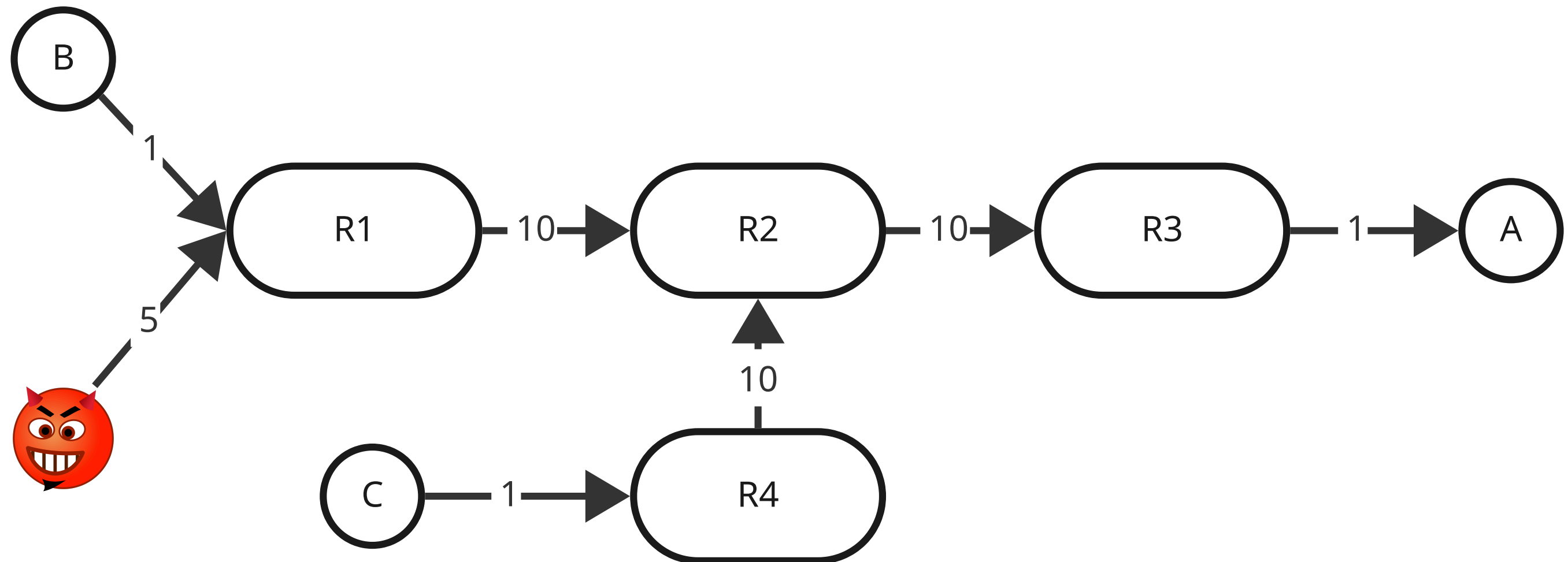
Ingress Filtering

Inspect incoming packets at the edge of a network (e.g., at a router or firewall).
It checks whether the source IP address of a packet is legitimate.

Involves verifying that the source IP address of the incoming packet is consistent with the network's policies or routing tables.
E.g. unicast reverse path

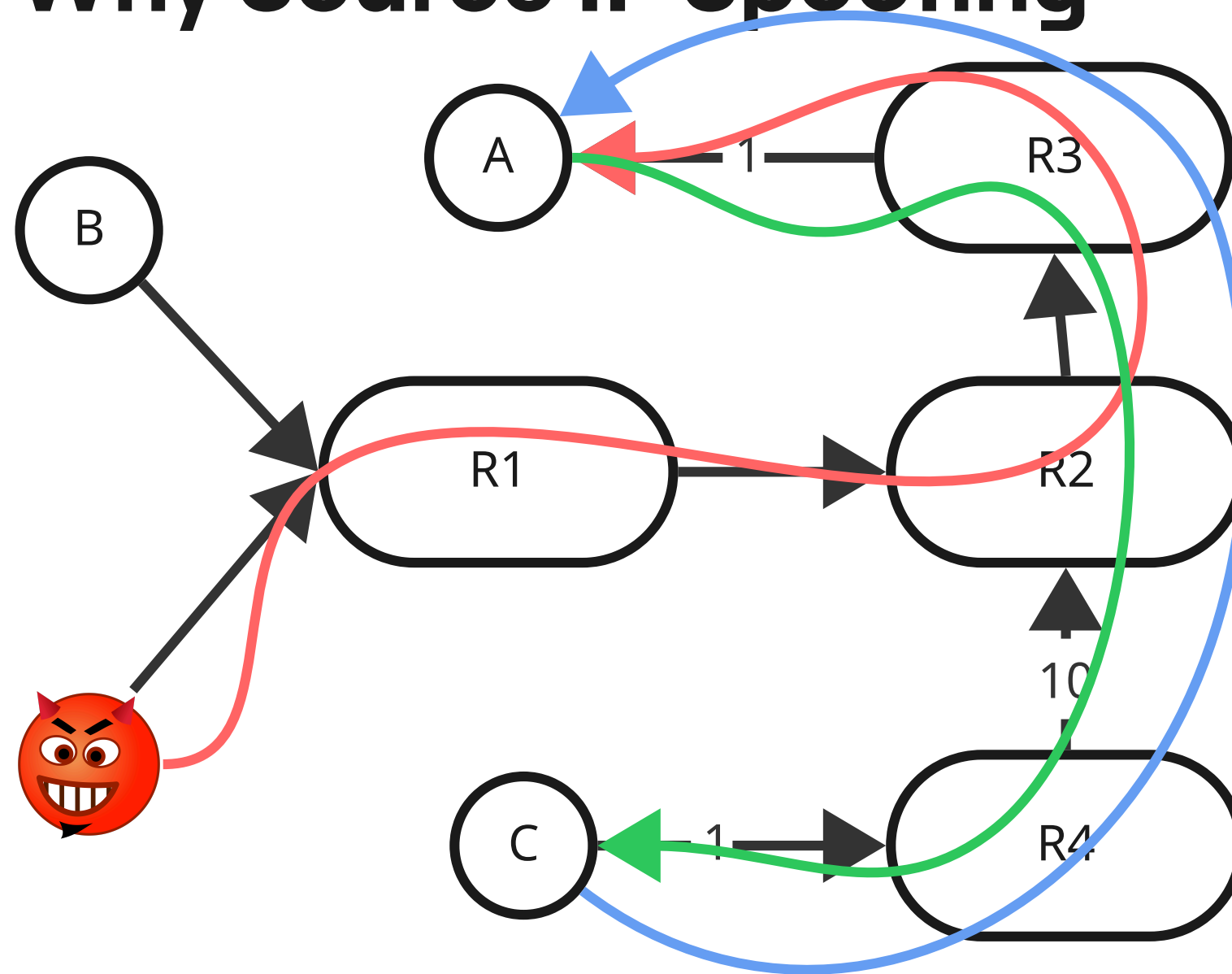
Implemented at the boundaries of a network – on routers or firewalls

ICMP flood



Source Address Spoofing: attacker can send packets with srcIP=C

Why Source IP Spoofing



Hide attacker identity

Avoid **back traffic**

ICMP Answers increase the victim's congestion

However ICMP Answers can be used to identify attacks (by R4, R2, and C)

Network Flooding attacks

Require more bandwidth of the victim

ICMP flood

Use ICMP packets: e.g. echo requests typically allowed through, some required

UDP flood

DNS, PTP, VoIP

TCP SYN flood

Use TCP syn (connection request) packets

Why connectionless protocols?



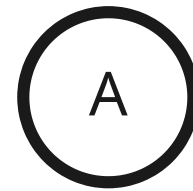
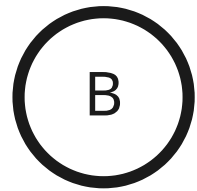
TCP Syn Spoof Attack

attack ability of a server to **respond to future TCP connection** requests

overflow tables used to manage them

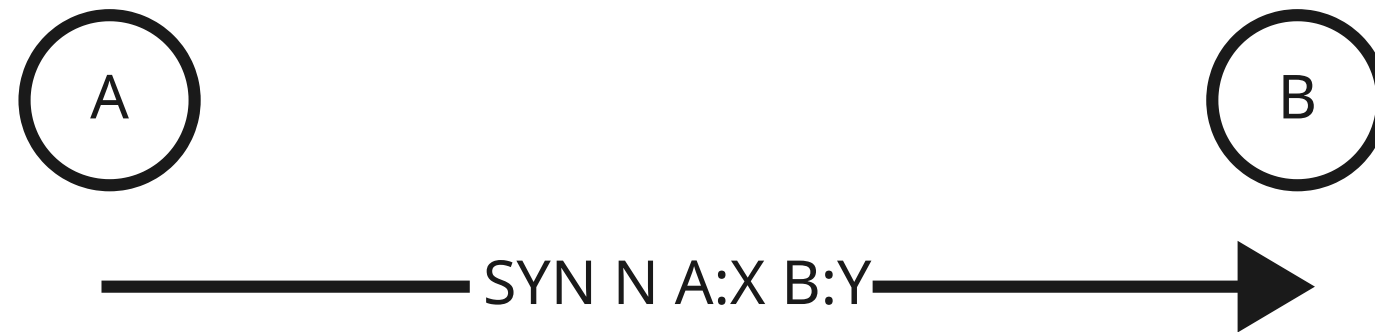
possible even if the attacker has **less bandwidth** than victim

3 way TCP handshake recap



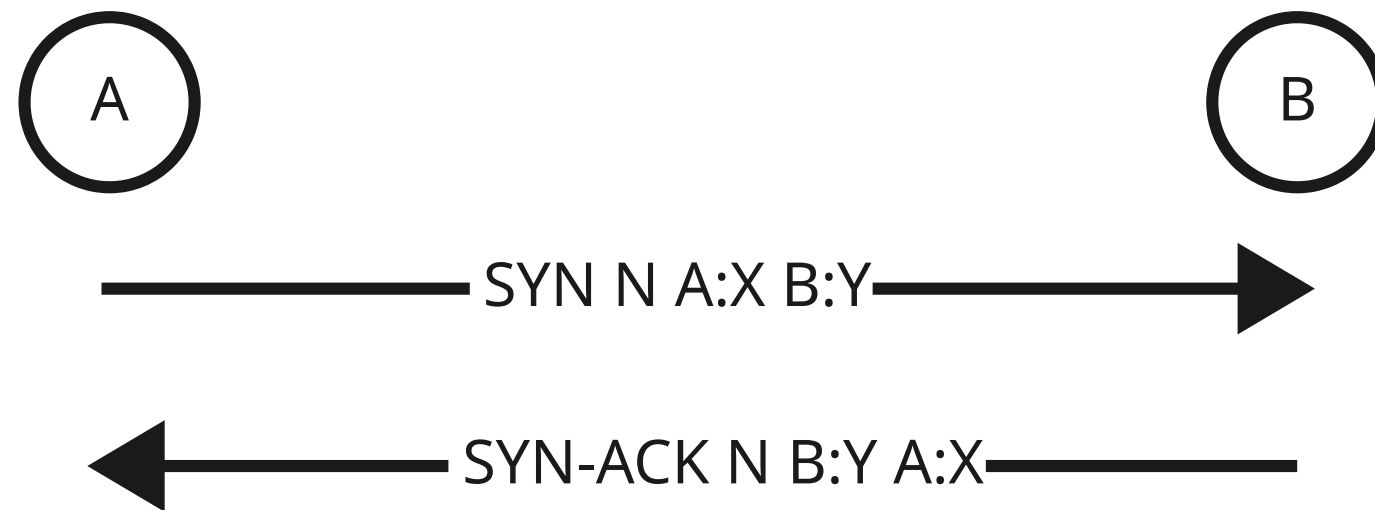
Connection Table

3 way TCP handshake recap



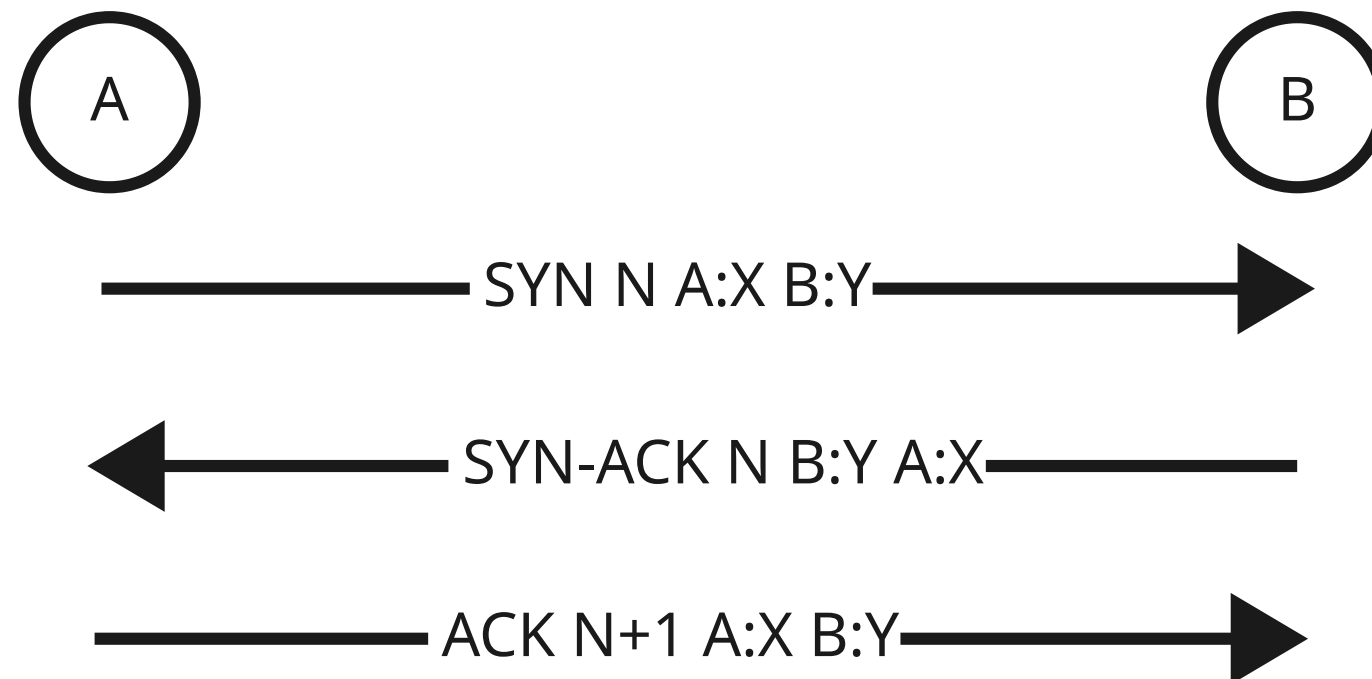
Connection Table
...
A:X B:Y N pending

3 way TCP handshake recap



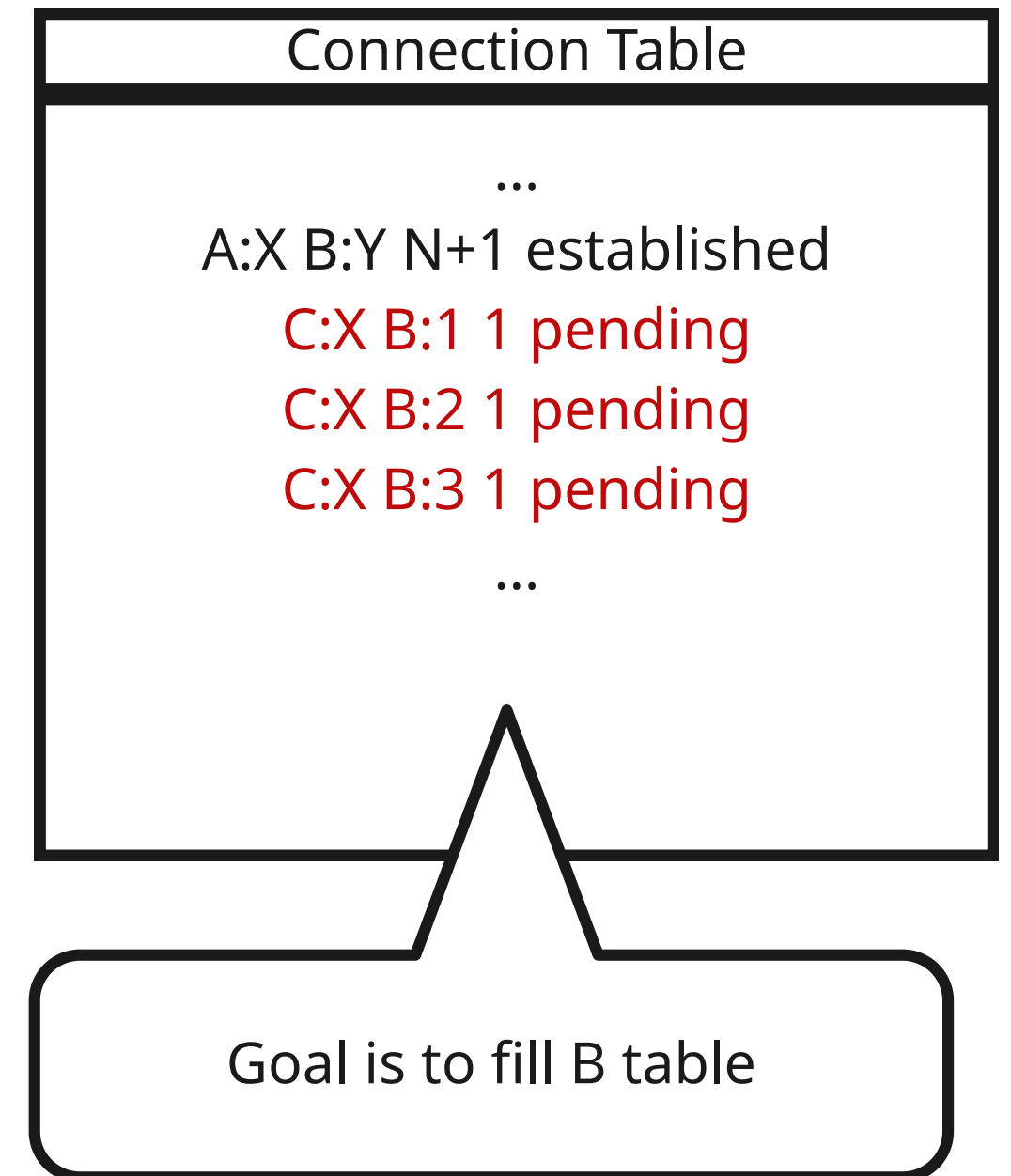
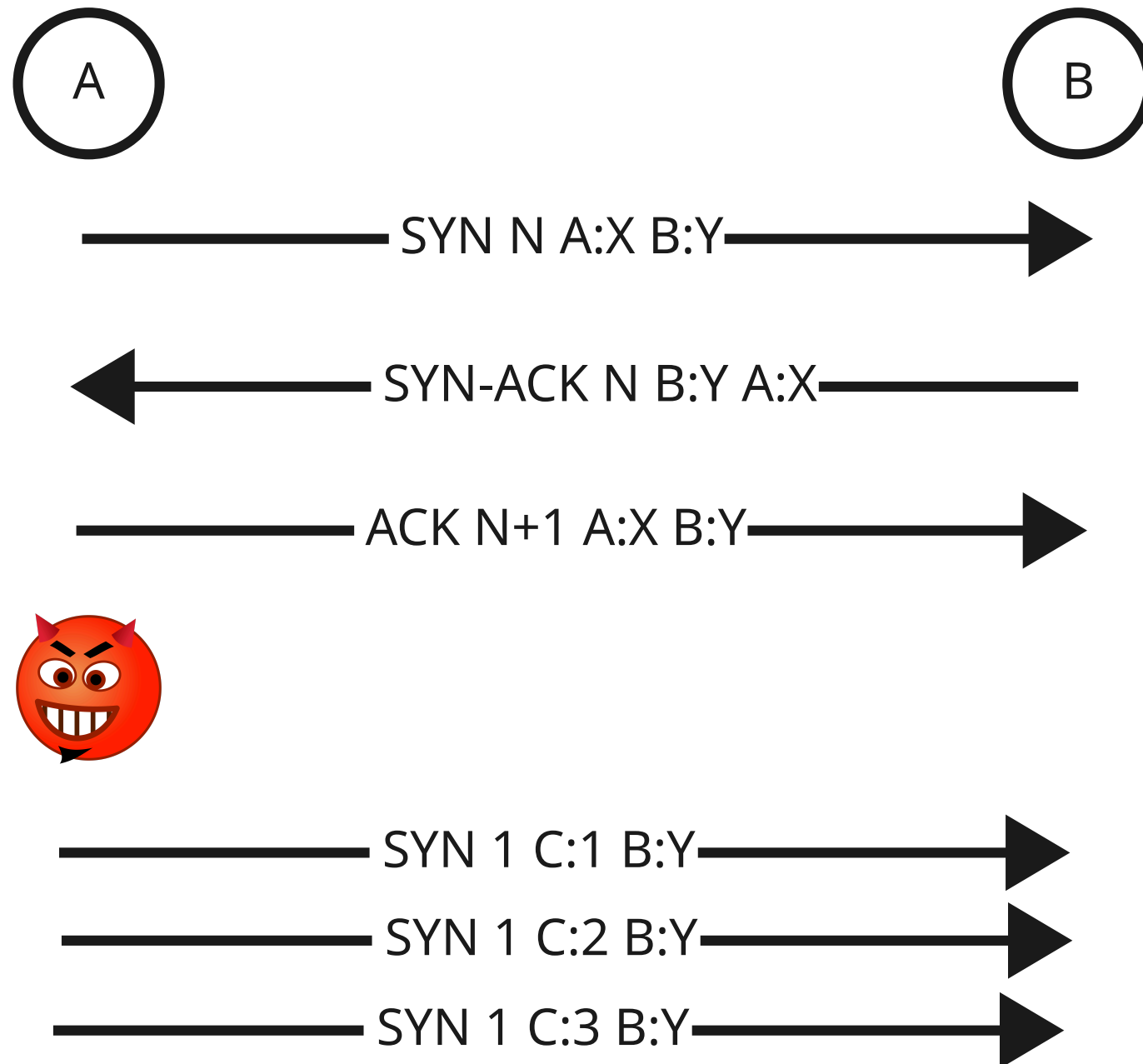
Connection Table
...
A:X B:Y N pending

3 way TCP handshake recap



Connection Table
...
A:X B:Y N+1 established

3 way TCP handshake recap



TCP Syn Spoof Defences

Time out pending connections

Limit number of connections per IP

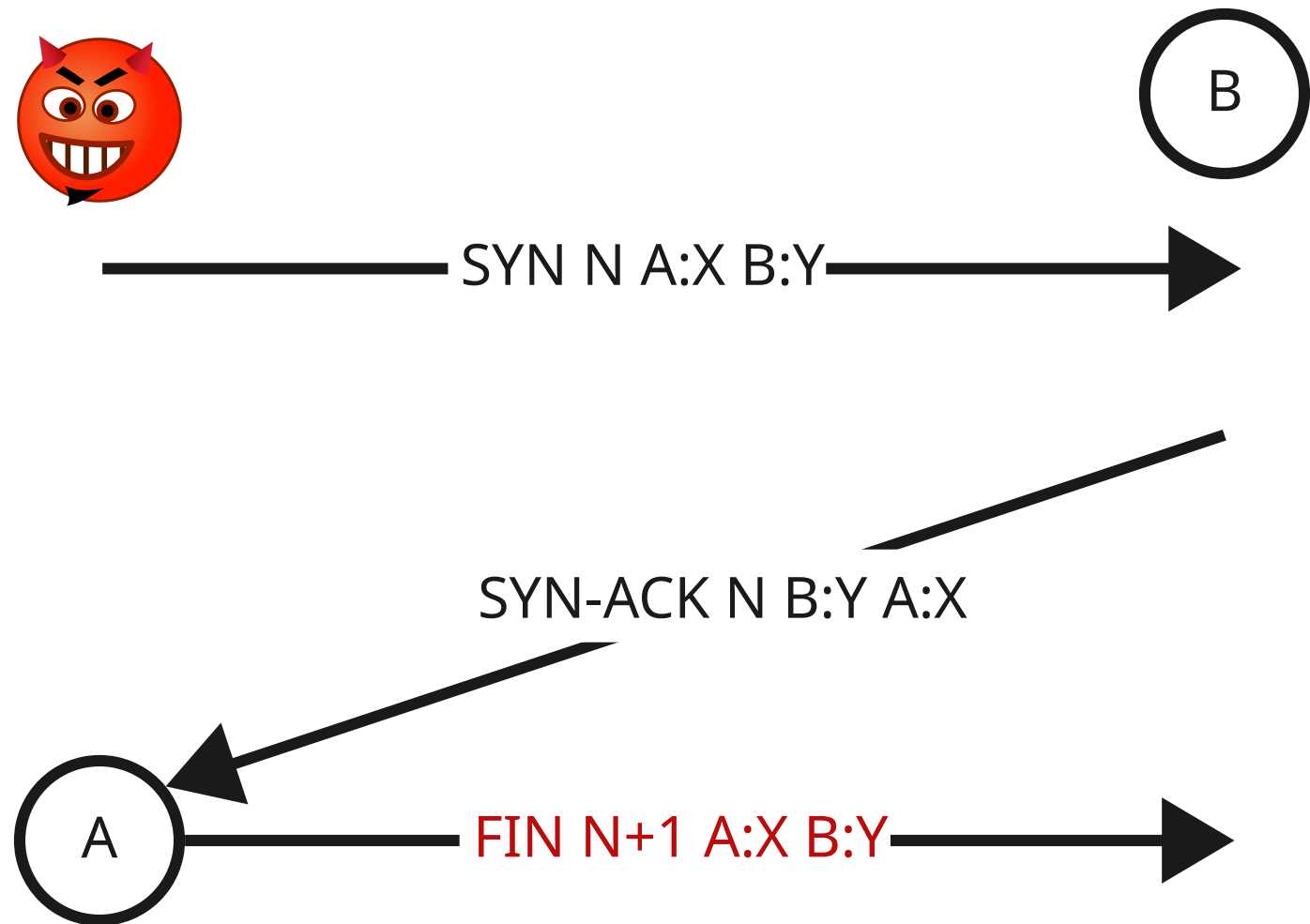
TCP Cookies

Send table row (encrypted and mac-ed) in TCP header

Receive the table row back later

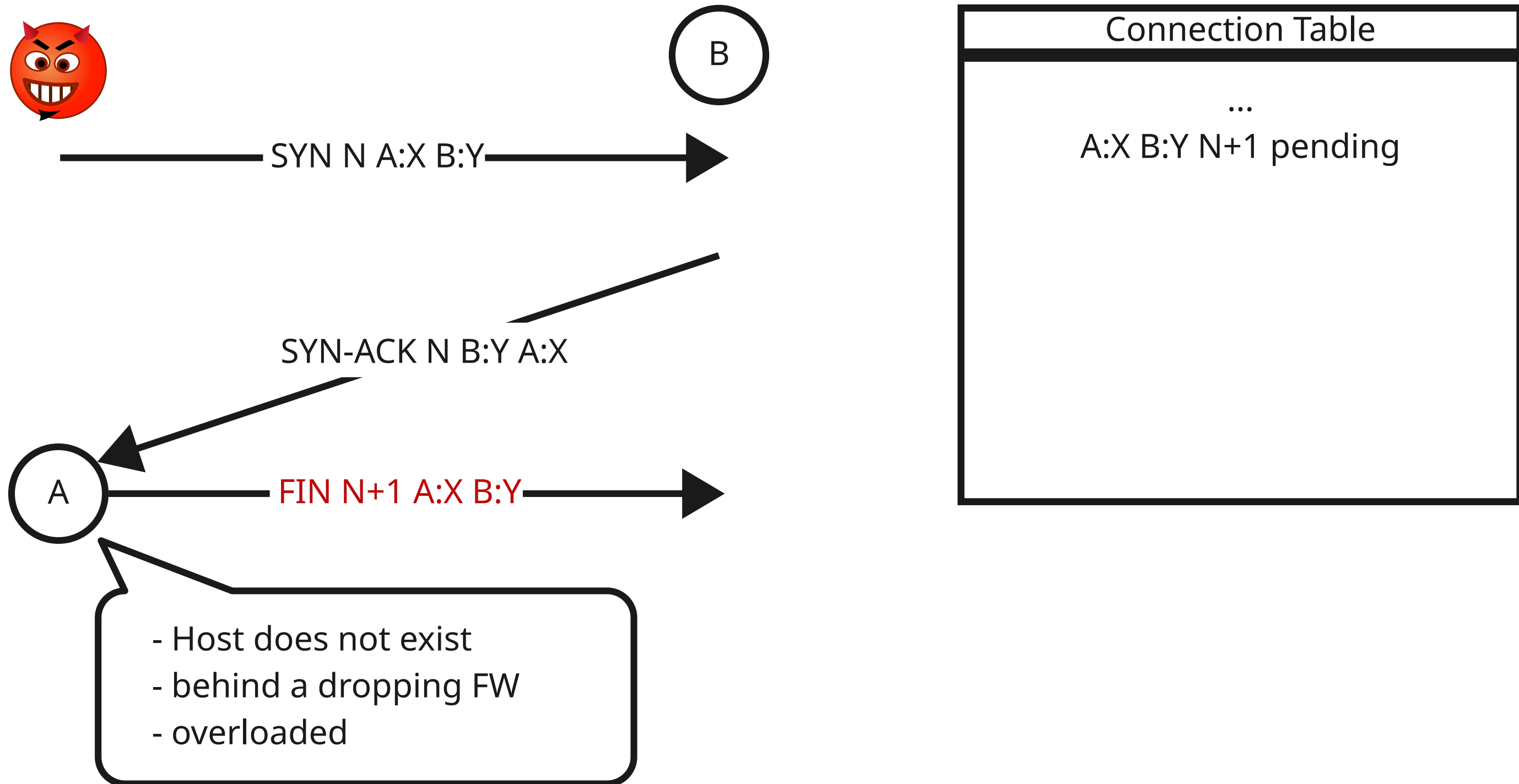
(At least for SYN-ACK / ACK)

TCP Syn Spoof via IP spoofing



Connection Table
...
A:X B:Y N+1 pending

TCP Syn Spoof via IP spoofing



Distributed DOS (DDOS)

Single host has limited volume

Multiple systems allow much higher traffic volumes

Distributed Denial of Service (DDoS) Attack

Often compromised PCs / workstations

Zombies/bots with backdoor programs installed forming a botnet

MIRAI (21 October 2016) Internet-of-Things' (IoT) botnet

Vulnerable DVRs, surveillance cameras, ...

500,000 compromised IoT devices, 665 Gbps

Multiple DOS attacks: SYN-flooding, UDP flooding, ...

Target Internet services firm Dyn (DNS servers)

Affected Twitter, Reddit, Spotify, SoundCloud, WhatsApp

DDOS as a service

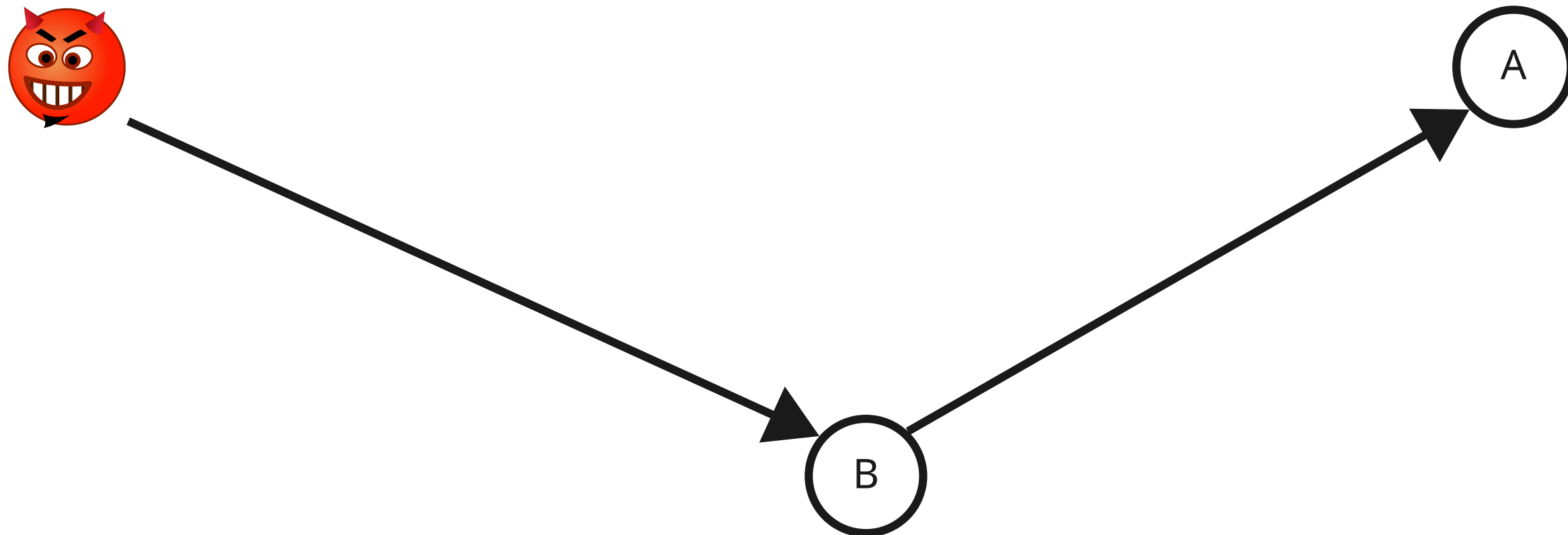
Using tools marketed for testing the resilience of a network ("stress testing")

Using botnet (and crypto currencies)

Reflection attacks

Various connectionless protocols: ICMP, DNS, TCP-SYN

Use spoofed src address

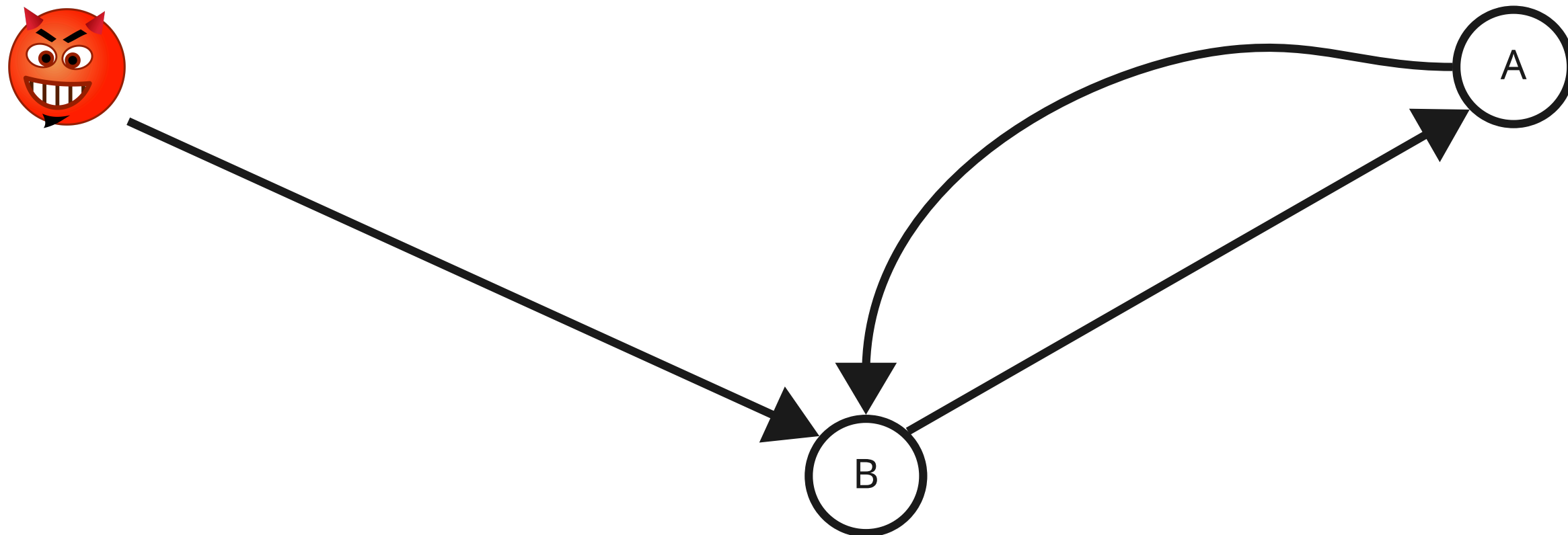


Reflection attacks

Various connectionless protocols: ICMP, DNS, TCP-SYN

Use spoofed src address

Loops: e.g. echo service

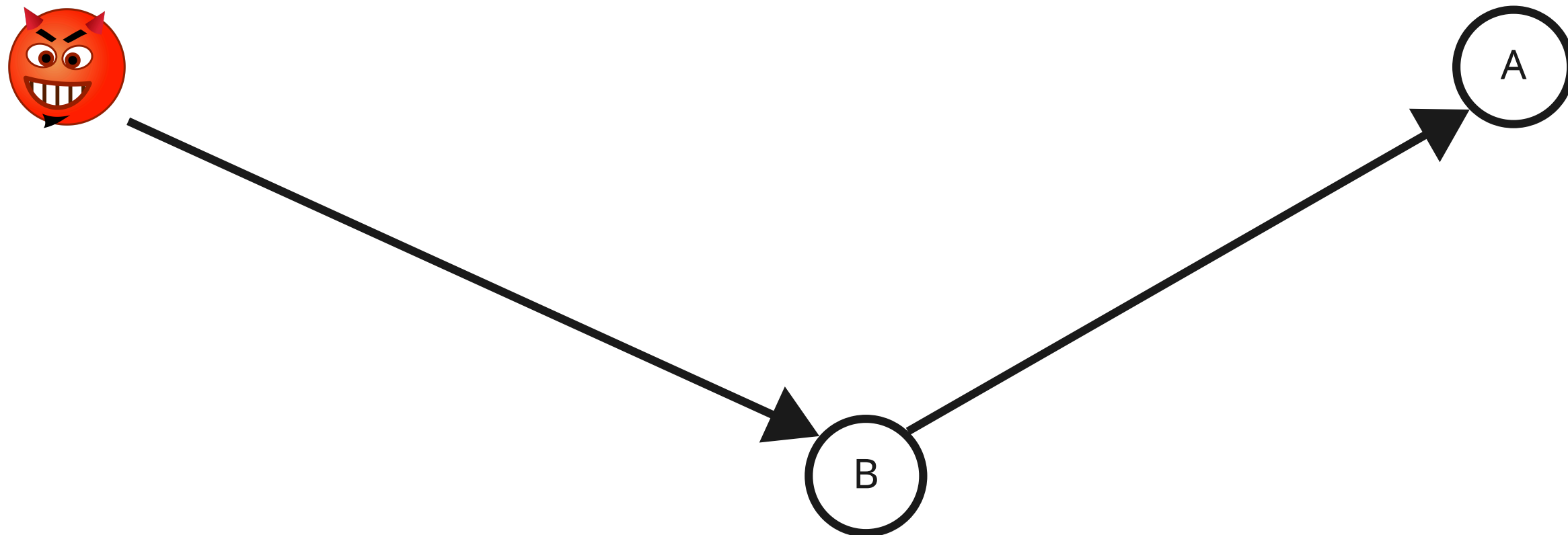


Amplification attacks

Various connectionless protocols: ICMP, DNS, TCP-SYN

Use spoofed src address

DNS request: 60 bytes, DNS response up to 4 Kb

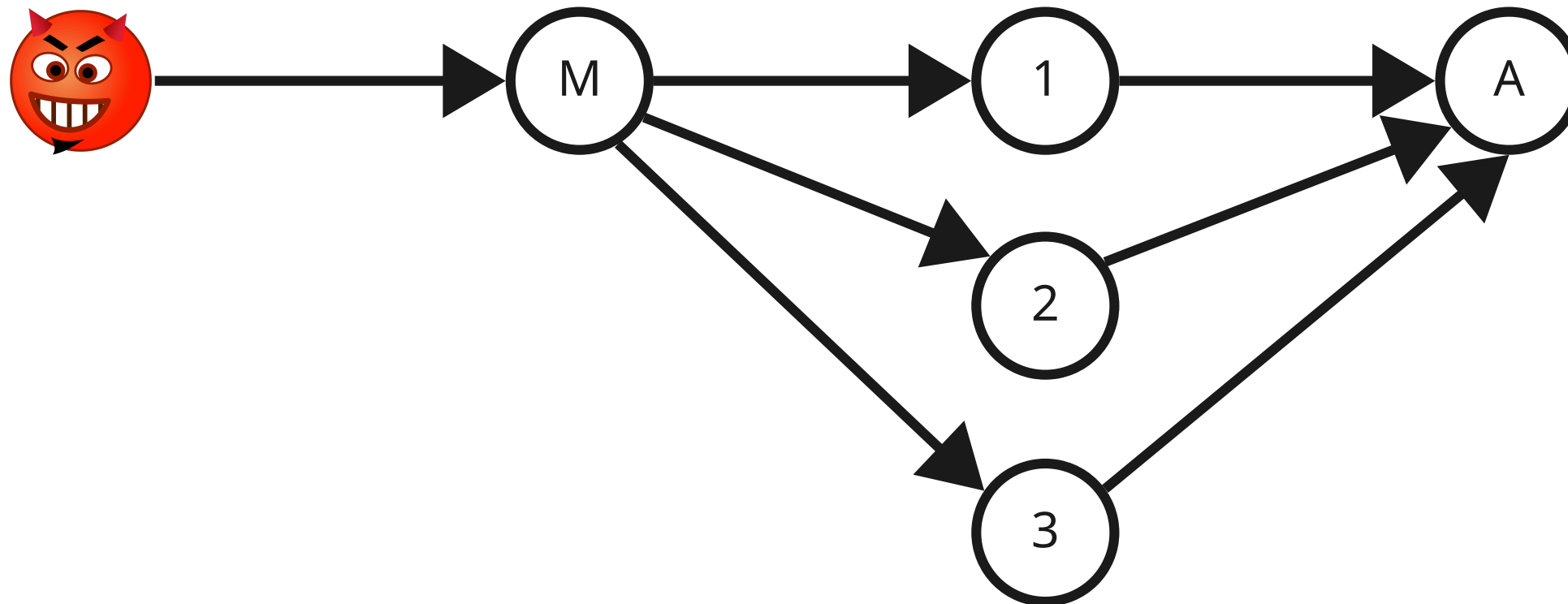


Amplification attacks

Various connectionless protocols: ICMP, DNS, TCP-SYN

Use spoofed src address

Using broadcast – multicast



BGB security

2021 October 4th Facebook outage due to a misconfiguration of BGP

Advertisement of a route
to a non-existing (or dropping) AS
to a congested AS
to a curious AS (see US Prism)

BGP RPKI To publish and validate critical routes

Software DOS

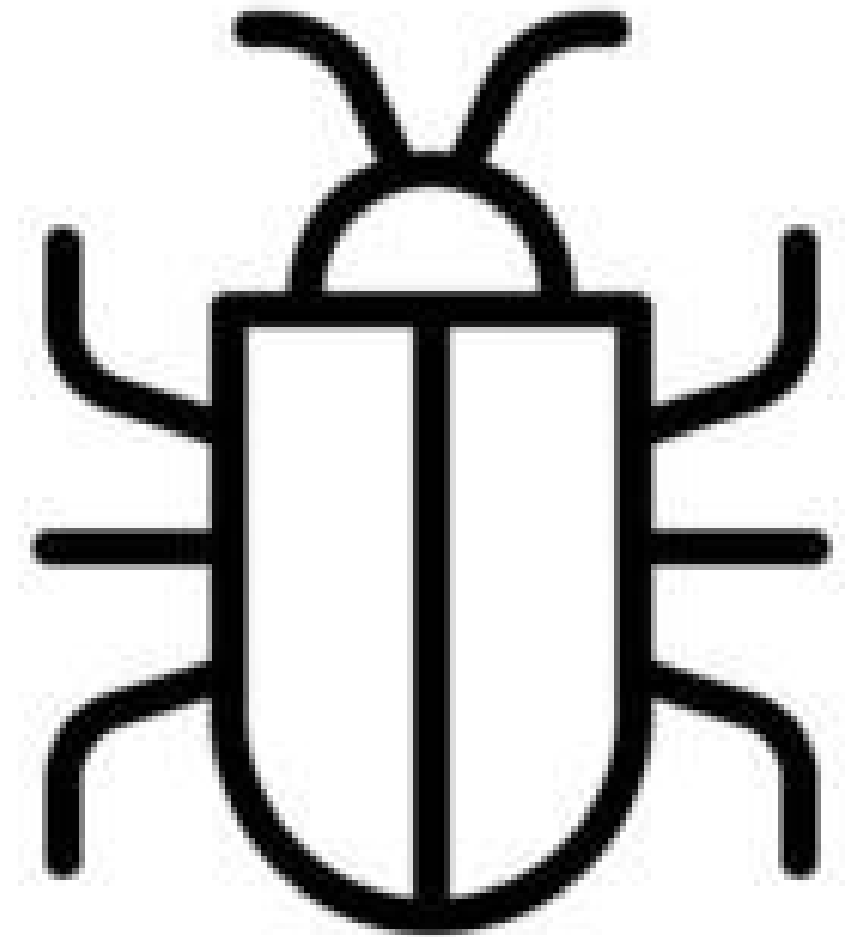
Based on SW vulnerabilities

Buffer overflows

Unhandled exceptions

Memory leaks

Resource leaks



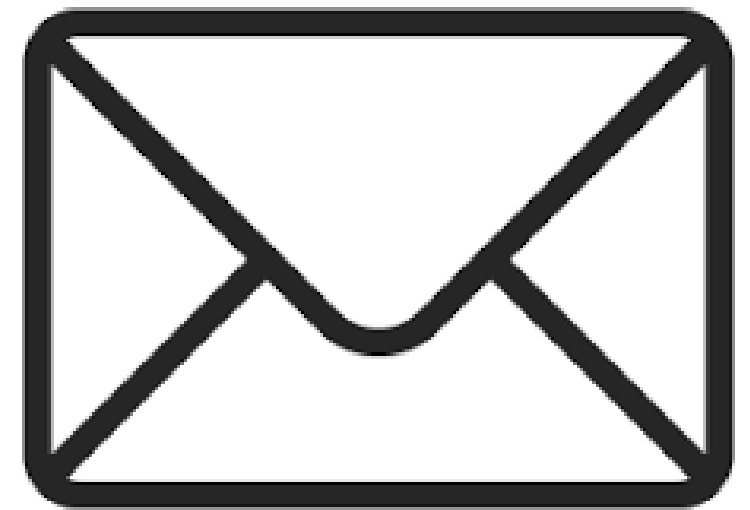
Email DOS

alice@kth.se and **bob@chalmers.se** set vacation messages
(i.e., automatic response)

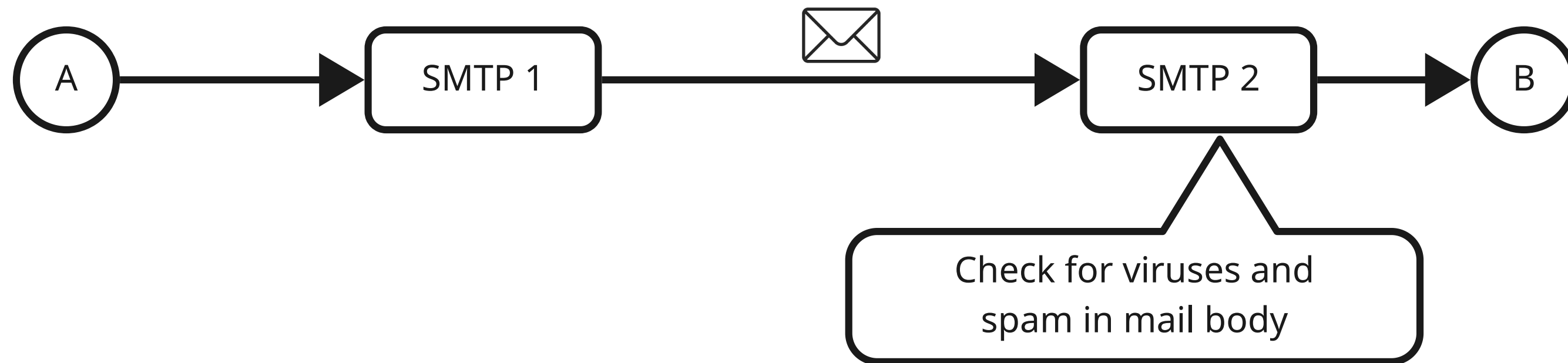
charlie@uu.se sends a (non signed) email with spoofed address
from: alice@kth.se
to: bob@chalmers.se
subject: hello
...

Chalmers's mail server sends an automatic reply to Alice
KTH's mail server sends an automatic reply to Bob
...

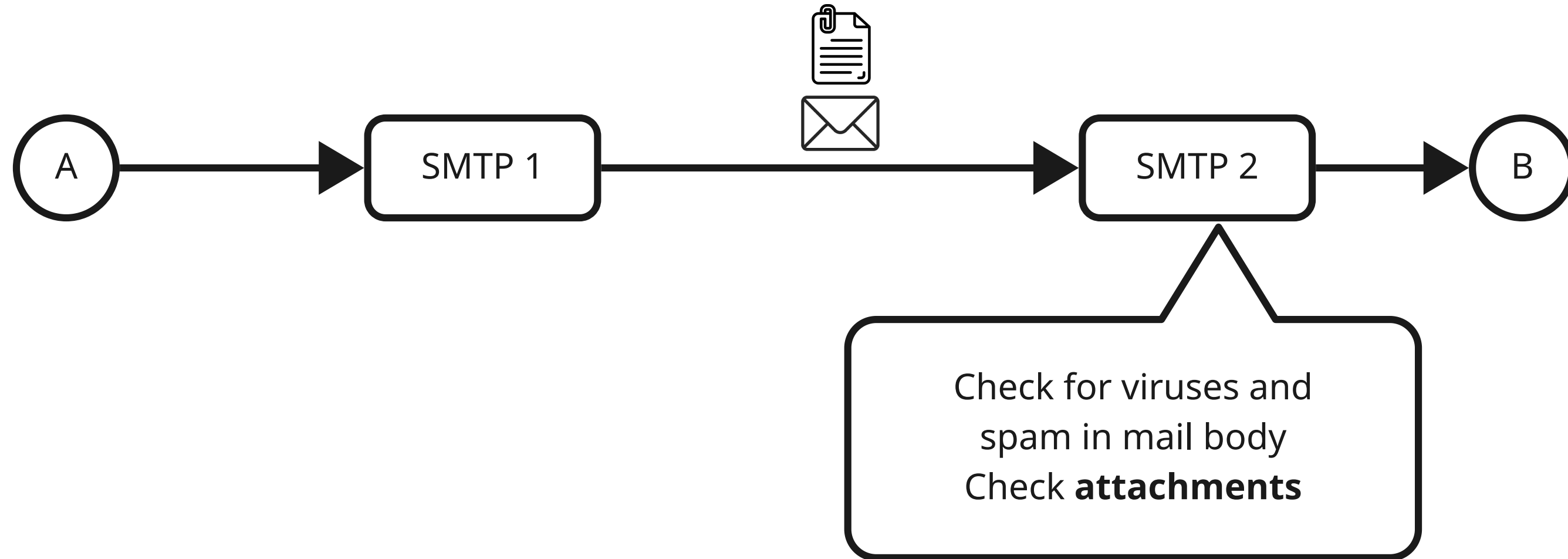
Until one of the two mailboxes (or SMTP queues) is full
Countermeasure: Use **Auto-Submitted** tag



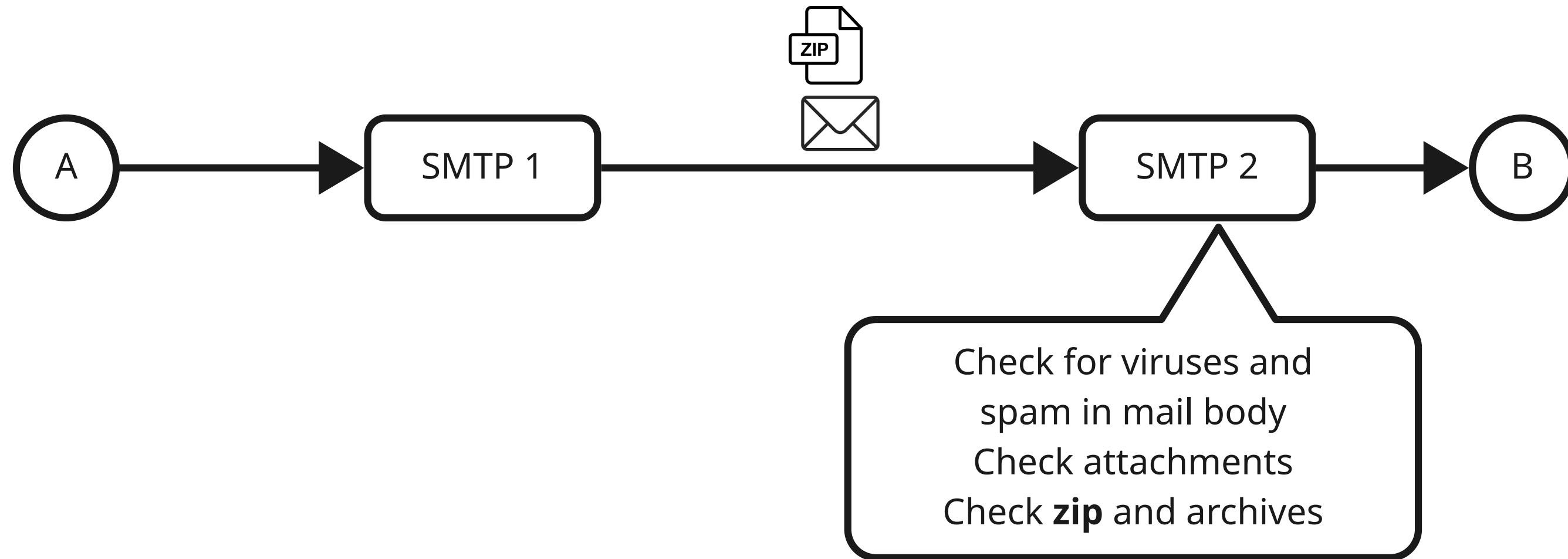
DOS on e-mail filters



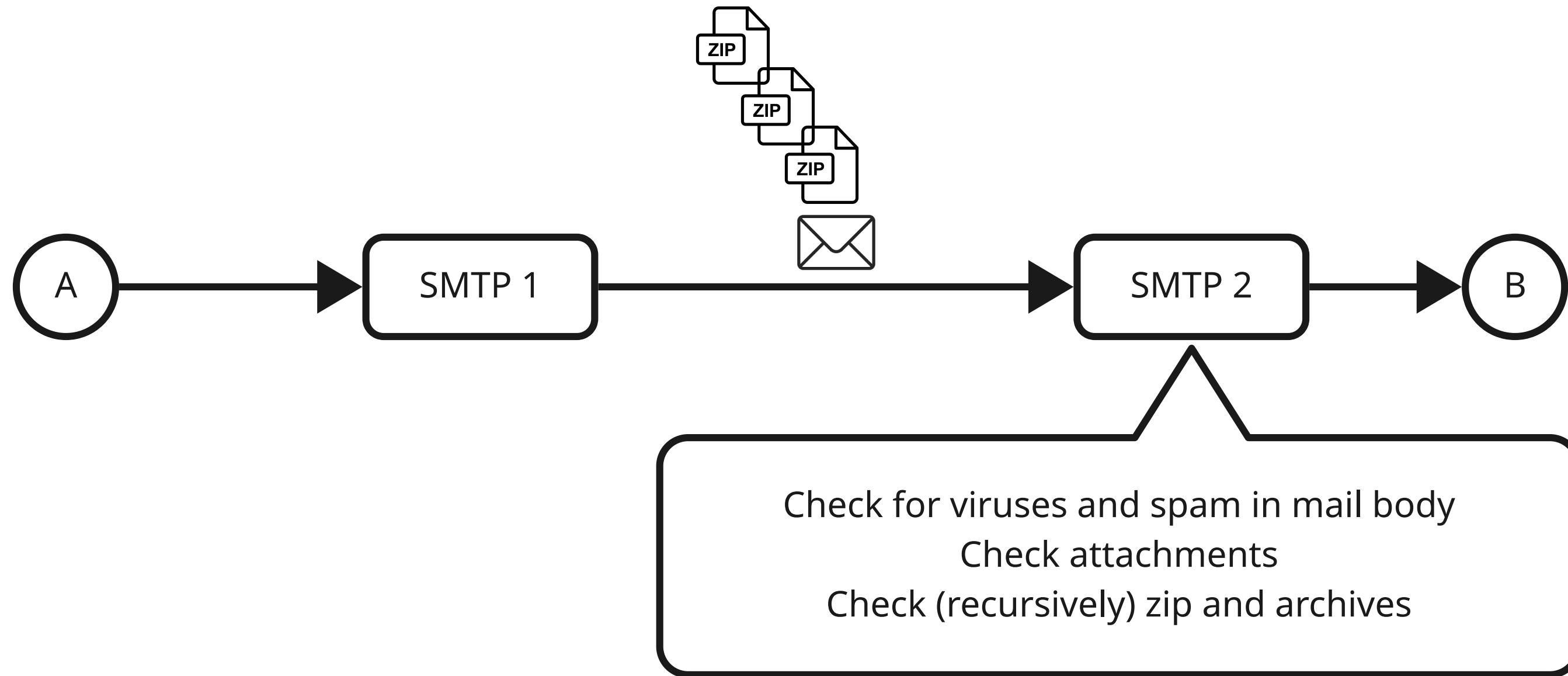
DOS on e-mail filters



DOS on e-mail filters



DOS on e-mail filters



DOS on e-mail filters

Infinite **recursive ZIP**

Bombs

gzip: 100 GB, 97 MB compressed, 1000:1 ratio

bzip2: 100 GB, 69 KB, 1.6×10^6 :1

PNG image: 19000 x 19000, 1-bit (45 MB) expand in 24-bit color to 1 GB, 44 KB compressed, 1000:1

Web DOS

Target supporting services e.g. DNS

Slowloris

Open TCP connection

Infinitely (and slowly) send HTTP headers

Consume the thread pool and connections of server



Intrusion Detection Systems IDS

IDS example: SMTP

TCP/IP vulnerabilities

SYN flood/spoofing (packet filtering)
external attacker using src port 25 (stateful inspection)

SMTP vulnerabilities

sendmail buffer overflow (SELinux)

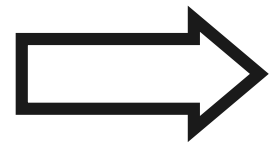
Mime vulnerabilities

PNG buffer overflow (SMTP IPS/antivirus)

User vulnerabilities

Phishing (Train users/SMTP IPS/antispam on server and client)

CWE Most Dangerous Software Weaknesses



1

Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')

[CWE-79](#) | CVEs in KEV: 3 | Rank Last Year: 2 (up 1) ▲

2

Out-of-bounds Write

[CWE-787](#) | CVEs in KEV: 18 | Rank Last Year: 1 (down 1) ▼

3

Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')

[CWE-89](#) | CVEs in KEV: 4 | Rank Last Year: 3

4

Cross-Site Request Forgery (CSRF)

[CWE-352](#) | CVEs in KEV: 0 | Rank Last Year: 9 (up 5) ▲

5

Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal')

[CWE-22](#) | CVEs in KEV: 4 | Rank Last Year: 8 (up 3) ▲

6

Out-of-bounds Read

[CWE-125](#) | CVEs in KEV: 3 | Rank Last Year: 7 (up 1) ▲

SQL Injection



SQL Injection

Defense Check that user input is not interpreted as a query operations

```
SELECT * FROM students WHERE name="%1";
```

Input a"); DROP TABLE students; --

```
SELECT * FROM students WHERE name="a"; DROP TABLE students; --";
```

Input a" OR ""="

```
SELECT * FROM students WHERE name="a" OR ""="";
```

Types of detectors

Three types of detectors

Network Intrusion Detection System (NIDS)

Host-based Intrusion Detection System (HIDS)

Log based Intrusion Detection System

The main difference is where the detector is deployed

Network Intrusion Detection System (NIDS)

A detector installed on the network, between the local network and the rest of the Internet

Monitors network traffic to detect attacks

Maintains state for each connection (Since majority of applications use TCP)

Often application proxies

(i.e., can deep inspect packets and sequence of packets)

Packet Filtering Firewall

applies rules to packets processed by firewall

in: **ingress** (INPUT chain in iptables)

out: **egress** (OUTPUT chain in iptables)

(FORWARD chain in iptables)

based on information in **packet header**

src/dest IP addr & port, IP protocol, interface

typically a list of rules of matches on fields

if match rule says if forward or discard packet

two default policies:

discard: prohibit unless expressly permitted (iptables -P INPUT DROP)

more conservative, controlled, visible to users

forward: permit unless expressly prohibited (iptables -P INPUT ACCEPT)

easier to manage use but less secure

Packet Filtering Firewall

Block a specific ip-address X

```
iptables -A INPUT -s "X" -j DROP
```

Allow Ping from inside to outside

```
iptables -A OUTPUT -p icmp --icmp-type echo-request -j ACCEPT
```

```
iptables -A INPUT -p icmp --icmp-type echo-reply -j ACCEPT
```

Allow loopback access

```
iptables -A INPUT -i lo -j ACCEPT
```

```
iptables -A OUTPUT -o lo -j ACCEPT
```

Allow all packets **from internal** network (eth0) **to** reach **external** network (eth1).

```
iptables -A FORWARD -i eth0 -o eth1 -j ACCEPT
```

Allow outbound DNS

```
iptables -A OUTPUT -p udp -o eth0 --dport 53 -j ACCEPT
```

```
iptables -A INPUT -p udp -i eth0 --sport 53 -j ACCEPT
```

Stateful Inspection Firewall

reviews packet header information but also keeps info on TCP connections

stateful inspection packet firewall tightens rules for TCP traffic using a TCP connection table

Stateful Inspection Firewall

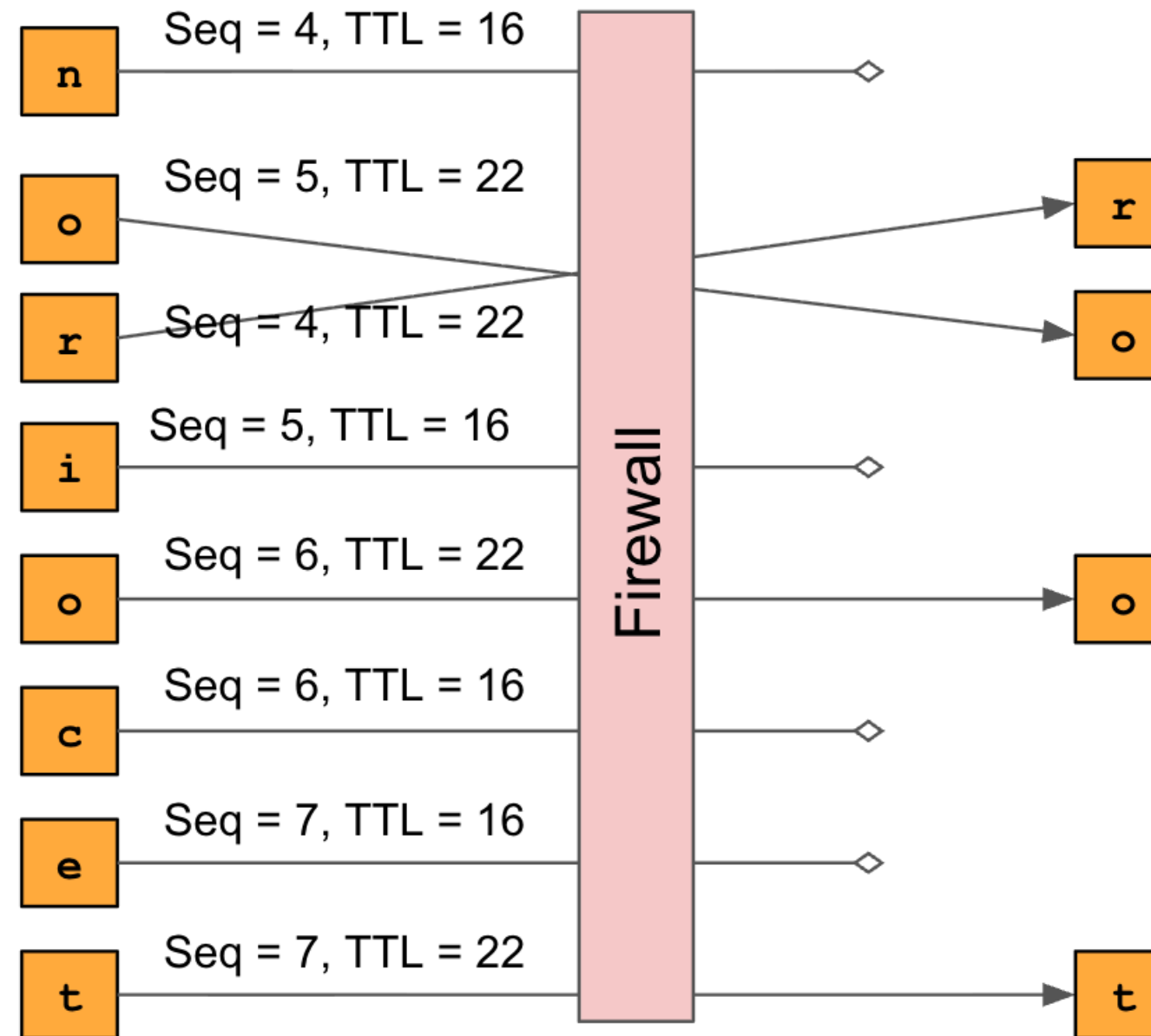
Allow ALL incoming SSH

```
iptables -A INPUT -i eth0 -p tcp --dport 22  
-m state --state NEW,ESTABLISHED -j ACCEPT  
iptables -A OUTPUT -o eth0 -p tcp --sport 22  
-m state --state ESTABLISHED -j ACCEPT
```

Allow outgoing SSH only to a specific network

```
iptables -A OUTPUT -o eth0 -p tcp -d 192.168.101.0/24 --dport 22  
-m state --state NEW,ESTABLISHED -j ACCEPT  
iptables -A INPUT -i eth0 -p tcp --sport 22  
-m state --state ESTABLISHED -j ACCEPT
```

Limitations: how to drop packets that form string root?



Bloom Filters

(represent large set of IPs)

Large number of hosts

2009 – 2010 Bredolab botnet 30M hosts

<http://www.projecthoneypot.org> Active stats

We represent $X \subseteq Y$ using an array A of n bits

We use k hash functions $H_{j \in [1 \dots k]} : Y \rightarrow [0 \dots n - 1]$

An empty (set) bloom filter has $A[i] = 0$ for every i

To add $x \in X$ we set $A[H_j(x)] = 1$ for $j \in [1 \dots k]$

$x \in X$ if $\forall j \in [1 \dots k]. A[H_j(x)] = 1$



False positive?
False negative?

Bloom Filters

Assuming A and B have the same n and hash functions

$$(A \cup B)[i] = A[i] \vee B[i]$$

Lossless: the resulting Bloom filter is the same as the Bloom filter created from scratch using the union of the two sets

$$(A \cap B)[i] = A[i] \wedge B[i]$$

false positive probability is at most the false-positive probability in one of the two Bloom filters

$$A \setminus \{x\}$$

Option is to use two bloom filters: one for added elements and one for removed
Circular list of Bloom filters (e.g., for recent TCP connections)

Application-Level Gateway (Application proxy firewall)

acts as a relay of application-level traffic

- user contacts gateway with remote host name

- authenticates themselves

- gateway contacts application on remote host and relays TCP segments between server and user

must have proxy code for each application

- may restrict application features supported

can inspect application specific information e.g. URLs

but have higher overheads

- ex: HTTP Proxy, SMTP chains, Local DNSs

NIDS Benefits

Cheap A single detector can cover a lot of systems

Easy to scale As the network gets larger, add computing power to the NIDS

Simple management Easy to install and manage a single detector

Transparent (End systems are unaffected)

- Doesn't consume any resources on end systems

- Useful for adding security on an existing system

Smaller trusted computing base (TCB)

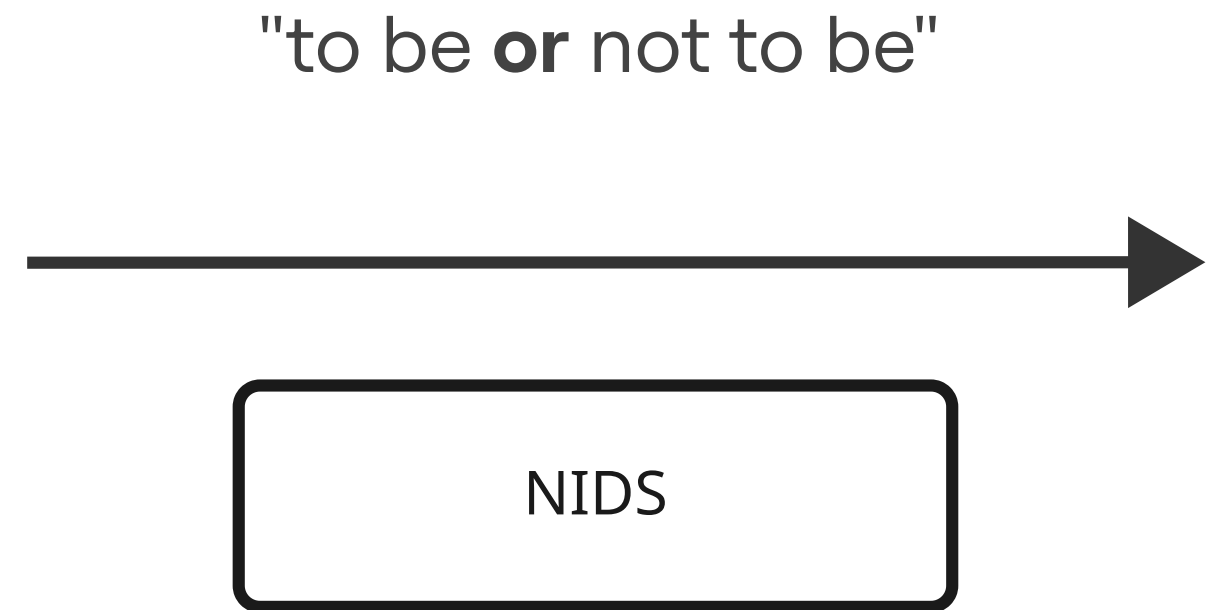
- Only the detector needs to be trusted

NIDS Drawbacks: Inconsistent Interpretation

This seems an SQL injection attack...
Maybe it should alert

What if the packet's TTL expires before it
reaches any end host?

Problem: What the NIDS sees doesn't
exactly match what arrives at the end
system



NIDS Drawbacks: Inconsistent Interpretation

This doesn't look like a SQL injection...
maybe it shouldn't alert

This input is using URL percent (i.e.,
base64) encoding.

If you decode it, you get **a" OR ""="!**

Problem: Inputs may be interpreted
differently between the NIDS and the end
system

a%22+OR+%22%22%3D%22



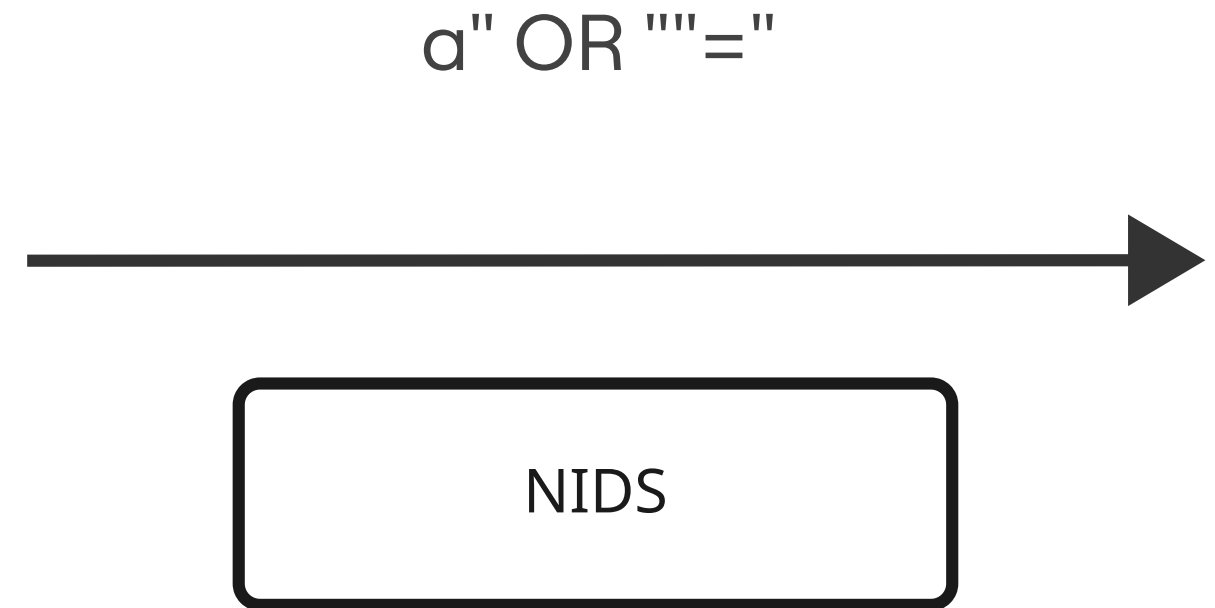
NIDS

NIDS Drawbacks: Inconsistent Interpretation

How this input is used by the application?

It's hard for the NIDS to know (e.g. imagine we are querying a source code repository)

Problem: Information needed to interpret correctly is missing



NIDS Drawbacks: encrypted traffic

Recall: TLS/SSL is end-to-end secure, so a NIDS can't read any encrypted traffic

One possible solution: Give the NIDS access to all the network's private keys

Now the NIDS can decrypt messages to inspect them for attacks

Problem: Users have to share their private key with someone else

Host-Based Intrusion Detection System (HIDS)

A detector installed on each end system

Embedded in the OS

As privileged process

As a middleware in the application stack

HIDS

Benefits

Fewer problems with inconsistencies or ambiguities: The HIDS is on the end host, so it will interpret packets exactly the same as the end host!

Works for encrypted messages

Can protect against non-network threats too (e.g. malicious user inside the network)

Performance may scale better than NIDS: one NIDS is more vulnerable to being overwhelmed than many HIDS

Drawbacks

Expensive: Need to install one detector for every end host

Evasion attacks are still possible (consider the problem of knowing how the application is using the input)

Log-Based Intrusion Detection System

Analyse log files generated by end systems

Example: Each night, run a script on the log files

Benefits

Cheap: Modern web servers often already have built-in logging systems

Fewer problems with inconsistencies or ambiguities: The logging system works on the end host, so it will interpret packets exactly the same as the end host!

Drawbacks

Unlike NIDS and HIDS, there is no real-time detection: attacks are only detected after the attack has happened

The attacker could change the logs to erase evidence of the attack (see forensics)

Detection Accuracy: errors

Two main types of detector errors

False positive Detector alerts when there is no attack

False negative Detector fails to alert when there is an attack

Detector accuracy is often assessed in terms of the rates of these errors

False positive rate (FPR) probability the detector alerts when there is no attack

False negative rate (FNR) probability the detector does not alert when there is an attack

Style of detectors

Four main styles of detectors

Signature-based detection

Specification-based detection

Anomaly-based detection

Behavioural detection

Bonus: Honeypots

Signature-based Detection

Flag any activity that matches the structure of a **known attack**

Signature-based detection is **blacklisting**

- Keep a list of patterns that are not allowed

- Alert if we see something on the list

Signatures can be at different network layers

- Example: TCP/IP header fields

- Example: URLs

- Example: Payload of the HTTP request

Signature-based Detection: Examples

Example: SQL Injection

We know that “**<something>--**” is often part of a SQL injection

Strategy: Alert if any request matches RegExp “***--**”

Example: phishing email

We know that phishing e-mail usually contain binary attachments

Strategy: Keep a list of know malware and alert if any mail contains a malware

Signature-based Detection: Trade Offs

Benefits

- Conceptually simple
- Very good at detecting known attacks
- Easy to share signatures and build up shared libraries of attacks
- Detect attacks that target vulnerabilities before patching SW

Drawbacks

- Won't catch new attacks without a known signature
- Might not catch variants of known attacks if the variant doesn't match the signature
- The attacker can modify their attack to avoid matching a signature

Specification-based Detection

Specify **allowed behaviour** and flag any behaviour that isn't allowed behaviour

Specification-based detection is **whitelisting**:

- Keep a list of allowed patterns

- Alert if we see something that is not on the list

Specification-based Detection: Examples

Example: SQL injection

We have a parameter representing user's name

We specify that only letters are allowed as input (except for Musk's childs)

Strategy: Alert if any request contains something other than a letter

Example: phishing email

Consider a mail requesting to report a grade in ladok

We know that for this kind of service, mail should only contain person number of user, code of course, and grade

Strategy: Flag mails that contain anything else

Specification-based Detection: Trade offs

Benefits

- Can detect new attacks we've never seen before

- If we properly specify all allowed behaviour, can have low false positive rate

Drawbacks

- Takes a lot of time and effort to manually specify all allowed behaviours

- May need to update specifications as things change

Anomaly-based Detection

Idea: Attacks look unusual

Develop a model of what normal activity looks like. Alert on any activity that deviates from normal activity.

Example: Analyse historical logs to develop the model

Similar to specification-based detection, but learn a model of normal behaviour instead of manually specifying normal behaviour

Anomaly-based Detection: examples

Example: SQL Injection

Analyse characters in requests and learn that “ never appears in normal requests

Strategy: Alert if any request contains “

Example: phishing emails

Let users report phishing emails

Learn that normal emails never contain string “prince of <country>”

Learn that normal emails to ladok@kth.se always come from a restricted set of addresses

Anomaly-based Detection: Trade Offs

Benefits

Can detect attacks we haven't seen before

Drawbacks

Can miss new and known attacks if they don't look unusual to our model

What if our model is trained on bad data (e.g. data with many of attacks)?

The false positive rate might be high (lots of non-attacks look unusual)

If we try to reduce false positives by only flagging the most unusual inputs, the false negative rate might be high (we miss slightly unusual attacks)

Behavioural Detection

Look for **evidence of compromise**

Unlike the other three styles, we are not scanning the input: We're looking at the actions triggered by the input

Instead of looking for the exploit, we're looking for the result of the exploit

Behaviours can themselves be analysed using blacklists (signature-based), whitelists (specification-based), or normal behaviour (anomaly-based)

Behavioural Detection: examples

Example: SQL Injection

- See if any unexpected amount of records are being accessed
- See if schema of the database changes

Example: phishing emails

- See if files are changed after accessing an email
- See if many emails are sent out
- See if the phone accesses bank-id

Behavioural Detection: Trade Offs

Benefits

- Can detect attacks we haven't seen before

- Can have low false positive rates if we're looking for behaviour that rarely occurs in normal programs (e.g. in the schema change example, there are probably no false positives!)

- Can be cheap to implement (e.g. existing tools to monitor system calls for a program)

Drawbacks

- Legitimate processes could perform the behaviour as well (e.g. accessing a password file)

- Only detects attacks **after** they've already happened

- Only detects successful attacks (maybe we want detect failed attacks as well)

- The attacker can modify their attack to avoid triggering some behaviour

Honeypots

A sacrificial system with no real purpose

No legitimate systems ever access the honeypot

If anyone accesses the honeypot, they must be an intruder

False positives: Legitimate systems mistakenly accessing the honeypot

Similar idea as **stack canaries** (we will see in memory safety)



Honeypots: examples

Hospitals

Employees should not read patient records

The hospital enters a honeypot record with a celebrity name

Catch any staff member who reads the honeypot record

Unsecured Bitcoin wallet

Leave an unsecured Bitcoin wallet on a system with a small amount of money

If the money is stolen, you know that someone has attacked your system!

Spamtrap

Create a fake email address that is never used for legitimate emails

If email gets sent to the address, it's probably spam!

Ransomware

Create a fake file that is never used

If a program reads that file, it's probably trying to encrypt it!

Honeypots: trade off

Benefits

- Can detect attacks we haven't seen before

- Can analyze the attacker's actions

 - Who is the attacker?

 - What are they doing to the system?

- Can distract the attacker from legitimate targets

Drawbacks

- Can be difficult to trick the attacker into accessing the honeypot

- Building a convincing honeypot might take a lot of work

- Should have few false positive e.g.,

 - the fake file may be accessed by the OS indexing features.

 - the user will see these useless files

These drawbacks matter less if the honeypot is aimed at automated attacks (e.g. the spam detection honeypot)

Attacks on IDSes

The IDS is a system with limited resources, so it is vulnerable to **DoS attacks!**

Exhaust the IDS's **memory**

IDS needs to track all ongoing activity

Attacker generates lots of activity to consume all the IDS's memory

Example: TCP SYN packets to force the IDS to keep track of too many connections

Exhaust the IDS's **processing power**

Example: If the IDS uses a hash table to keep track of connections, create hash collisions to trigger worst-case complexity

The IDS analyzes outside input, so it is vulnerable to **code injection** attacks!

Attacker supplies malicious input to exploit the IDS